Am9080 Instruction Execution

Machine cycles, T-states, bus timing, and a Verilog implementation roadmap

1. The execution hierarchy

The Am9080 executes instructions using sequential machine cycles and clock states. It still fetches, decodes, and executes instructions, but it is not a modern pipelined CPU with several instructions simultaneously progressing through independent pipeline stages.

Instruction cycle: The entire process of fetching and completing one instruction.

Machine cycle (M-cycle): One portion of an instruction, normally associated with a bus activity such as opcode fetch, memory read, memory write, or I/O.

Clock state (T-state): One processor clock period within a machine cycle; signals and internal actions are coordinated at this granularity.

Instruction cycle → M1, M2, … → T1, T2, T3, [T4, T5 or wait states]

2. What the T-states do

State	Typical activity
T1	Place address and machine-cycle status on buses; assert SYNC.
T2	Establish read/write signaling and determine whether the external device is ready.
TW	Optional wait period when READY is not asserted.
T3	Complete the bus transfer.
T4/T5	Additional internal operations when required by the instruction.

These are typical roles, not universal fixed pipeline stages. Exact signal edges and per-instruction sequences must be checked against AMD timing diagrams.

3. Worked example: MVI A, 42H

Assume PC = 1000H, memory[1000H] = 3EH (opcode), and memory[1001H] = 42H (immediate byte). The instruction uses two machine cycles and seven T-states in the normal no-wait case.

Cycle	States	Conceptual action
M1: opcode fetch	T1–T4	Present 1000H; read opcode 3EH; decode and prepare operand read.
M2: memory read	T1–T3	Present 1001H; read immediate byte 42H; load A.

Result: A = 42H, PC = 1002H. This is a conceptual operation timeline; use the original manual to determine exact sampling and update edges.

4. Compare with MOV A, B

MOV A, B transfers data between internal registers. It requires only an opcode-fetch machine cycle (five T-states for this instruction), because no extra operand byte or external data read is needed. Instruction length, number of machine cycles, and number of T-states are distinct properties.

5. Designing the controller in Verilog

A useful RTL decomposition is an instruction decoder, a machine-cycle sequencer, a T-state sequencer, and a bus-control/datapath control generator. Track the current opcode, machine-cycle position, machine-cycle type, and T-state. M2 is a position in the sequence, not a universal operation: it can be a memory read, write, or another operation depending on the instruction.

A first exercise is to implement a memory-read controller with T1 → T2 → (TW as needed) → T3, then add opcode fetch and memory write. Verify address stability, status output, READY handling, and transfer completion in a waveform viewer. Model the two-phase original clock carefully when moving from conceptual timing to exact external compatibility.

6. Research materials

AMD 8080A/9080A MOS Microprocessor Handbook (1977) — primary source

Start with Chapter 2: functional description, machine-cycle timing, status outputs, and interface signals.

https://www.bitsavers.org/components/amd/_dataBooks/1977_AMD_8080A_9080A_MOS_Microprocessor_Handbook.pdf

Intel 8080 Microcomputer Systems User's Manual

Read the processor cycle, state transition sequence, and state-associated activities.

https://manualzz.com/doc/6944712/intel-8080-microcomputer-central-processing-unit-user-manual

Intel SBC-80/10 Hardware Reference Manual

Use as a supplementary example of how the CPU communicates with external hardware.

https://manuals.plus/m/831cd83b02b99148868cac6a318451d3f13551ee807d625cd93bed6d99bdc1dc

Intel MCS-80/85 User's Manual

Useful for instruction descriptions, timing counts, and flag behavior.

https://www.bitsavers.org/components/intel/MCS80/MCS80_85_Users_Manual_Jan83.pdf

7. Suggested reading sequence

First understand the machine-cycle hierarchy. Then trace a memory read and opcode fetch state by state in the AMD diagrams. Next compare MVI, MOV, LDA, CALL, and RET. Finally implement one bus cycle and validate it against a testbench before attempting the full instruction sequencer.